

// javascript

```
db.users.insertMany([
  {
    _id: ObjectId(),
    name: "Tarun Patidar",           // string
    age: 22,                         // number
    salary: 45500.75,                // double
    isActive: true,                  // boolean
    phone: "9876543210",             // phone number
    aadhaar: "123456789012",         // aadhaar number
    password: "Tarun@1234Abcd$X",    // password
    dob: new Date("2004-08-15"),     // date
    middleName: null,                 // null
    tempField: undefined,             // undefined

    address: {                       // embedded document
      city: "Indore",
      state: "Madhya Pradesh",
      pincode: 452010
    },

    skills: ["C++", "MongoDB", "Node.js"], // array
  }
])
```

```
education: [                                     // array of embedded documents
  { level: "10th", year: 2018, percentage: 82.4 },
  { level: "12th", year: 2020, percentage: 78.6 }
],

extraInfo: { hobby: "Cricket" },                // schemaless field
profilePic: BinData(0, "U29tZURhdGE="), // binary data
createdAt: new Date()
},

{
  _id: ObjectId(),
  name: "Priya Verma",
  age: 24,
  salary: 52000.50,
  isActive: false,
  phone: "8123456789",
  aadhaar: "987654321098",
  password: "Priya#5678Xyzm@Q",
  dob: new Date("2002-03-20"),
  middleName: null,
```



```
// JavaScript
```

```
insertOne({ name: "Ravi",
```

```
  address: {
    city: "Bhopal",
    state: "Madhya Pradesh",
    pincode: 462001
  },
```

```
  skills: ["Java", "SQL"],
  education: [
    { level: "BTech", college: "LNCT", cgpa: 8.1 }
  ],
```

```
  extraInfo: { favoriteColor: "Blue", expectedCTC: 600000 },
  createdAt: new Date()
}
```

```
])
```

/C:/0a30b0e1-ec10-03ee-929e-0700e12b4e41

Default behavior

JavaScript

```
{ ordered: true }
```

Meaning

- Documents **sequence me insert** honge
- Agar beech me **error aa gaya**, to **uske baad wale documents insert nahi honge**

Example

JavaScript

```
db.users.insertMany(  
  [  
    { _id: 1, name: "Tarun" },  
    { _id: 1, name: "Priya" }, // duplicate _id -> error  
    { _id: 3, name: "Rahul" }  
  ],  
  { ordered: true }  
)
```



2) Unordered Insert

Syntax

```
</> JavaScript  
  
{ ordered: false }
```

Meaning

- Agar kisi ek document me error aaye, tab bhi **baaki valid documents insert hote rahenge**

Example

```
</> JavaScript  
  
db.users.insertMany(  
  [  
    { _id: 1, name: "Tarun" },  
    { _id: 1, name: "Priya" }, // duplicate _id -> error  
    { _id: 3, name: "Rahul" }  
  ],  
  { ordered: false }  
)
```



MongoDB BSON Data Types

[Upgr](#)

Data Type	BSON Type Number	Size	Use	
String	2	Variable	Name, Email, Address	
Int32	16	4 bytes	Age, Marks	
Int64 (Long)	18	8 bytes	Large IDs, Big Numbers	
Double	1	8 bytes	Decimal approximation	
Decimal128	19	16 bytes	Money, Salary	
Boolean	8	1 byte	true / false	
Date	9	8 bytes	DOB, CreatedAt	
Timestamp	17	8 bytes	Replication, Internal	
ObjectId	7	12 bytes	Default <code>_id</code>	

MongoShell command

ObjectId	7	12 bytes	Default <code>_id</code>	
Array	4	Variable	Skills, Subjects	
Object (Document)	3	Variable	Nested Documents	
Null	10	-	Null values	
Binary	5	Variable	Files, Images	
Regular Expression	11	Variable	Regex	
JavaScript	13	Variable	JS Code (Rare)	
MinKey	-1	-	Internal	
MaxKey	127	-	Internal	

SQL vs MongoDB	
SQL	MongoDB
INT	Int32
BIGINT	Int64 (Long)
FLOAT	Double
DECIMAL	Decimal128
VARCHAR	String
CHAR	String
TEXT	String
BOOLEAN	Boolean
DATE	

MongoShell command

▼	BOOLEAN	Boolean
	DATE	Date
	TIMESTAMP	Timestamp / Date
	BLOB	Binary
	Table	Collection
	Row	Document
	Column	Field

Safe version (pehle copy check karo, baad me old db delete karo)

JavaScript

```
use companyDB
```

```
db.getCollectionNames().forEach(function(collName) {  
  const docs = db[collName].find().toArray();  
  if (docs.length > 0) {  
    db.getSiblingDB("corporateDB")[collName].insertMany(docs);  
  }  
})
```

Dbir check karo:

⌘ JavaScript

```
db.orders.renameCollection("salesOrders")
```

5) payments → salaryPayments

⌘ JavaScript

```
db.payments.renameCollection("salaryPayments")
```

6) courses → trainingPrograms

⌘ JavaScript

```
db.courses.renameCollection("trainingPrograms")
```

Haan 👍

MongoDB shell me `updateOne()` ka syntax ye hota hai:

Syntax

⌘ JavaScript

```
db.collectionName.updateOne(  
  { condition },  
  { $set: { field: value } }  
)
```

1) employee ka city update karna

Agar employee ka naam **Tarun Patidar** hai aur uska city **Indore** karna hai:

</> JavaScript

```
db.employees.updateOne(  
  { name: "Tarun Patidar" },  
  { $set: { city: "Indore" } }  
)
```

2) student ki fees update karna

</> JavaScript

```
db.students.updateOne(  
  { name: "Rahul Sharma" },  
  { $set: { fees: 75000 } }  
)
```



Multiple fields ek saath update

JavaScript

```
db.employees.updateOne(
  { name: "Tarun Patidar" },
  { $set: { city: "Pune", salary: 95000, department: "Development" } }
)
```

Agar value increase karni ho (\$inc)

salary me 5000 add

JavaScript

```
db.employees.updateOne(
  { name: "Tarun Patidar" },
  { $inc: { salary: 5000 } }
)
```



Agar field remove karni ho (\$unset)

⌘ JavaScript

```
db.employees.updateOne(  
  { name: "Tarun Patidar" },  
  { $unset: { bonus: "" } }  
)
```

Result check karna

⌘ JavaScript

```
db.employees.find({ name: "Tarun Patidar" })
```

updateOne() vs updateMany()

- `updateOne()` → sirf 1 matching document update karega
- `updateMany()` → jitne bhi matching documents honge sab update karega

Tumhare database ke hisaab se examples

1) Indore ke sab students ka department update

JavaScript

```
db.students.updateMany(  
  { city: "Indore" },  
  { $set: { department: "CSE" } }  
)
```


8) Sab customers se likes field hata do

JavaScript

```
db.customers.updateMany(  
  {},  
  { $unset: { likes: "" } }  
)
```

Multiple fields ek saath update

9) Pune ke employees ka city aur department dono update

JavaScript

```
db.employees.updateMany(  
  { city: "Pune" },  
  { $set: { city: "Pune Maharashtra", department: "Software" } }  
)
```



Condition ke saath range update

10) Jinki salary 100000 se kam hai unka bonus increase

</> JavaScript

```
db.employees.updateMany(  
  { salary: { $lt: 100000 } },  
  { $inc: { bonus: 3000 } }  
)
```

Result check karna

</> JavaScript

```
db.employees.find({ department: "IT" })
```

ya



fdff

1. `deleteOne()` — ek document delete

</> JavaScript

```
db.firstCollection.deleteOne({  
  name: "Rahul"  
})
```

➔ `name: "Rahul"` wala ek matching document delete hoga.

2. `deleteMany()` — multiple documents delete

</> JavaScript

```
db.firstCollection.deleteMany({  
  city: "Indore"  
})
```

➔ `city = "Indore"` wale saare documents delete honge.



3. `findOneAndDelete()` — find + delete

JavaScript

```
db.firstCollection.findOneAndDelete({  
  name: "Rahul"  
})
```

➡ Ek matching document delete hoga aur **deleted document return** hoga.

4. `deleteById()` — Mongo Shell me direct method nahi hai ❌

Mongo Shell me `_id` se delete karne ke liye:

JavaScript

```
db.firstCollection.deleteOne({  
  _id: ObjectId("66c123abc456789012345678")  
})
```



1. `$and` ke saath `deleteMany()`

Suppose delete karna hai:

age 23 se 55 ke beech **AND** city = nalkheda

`</>` JavaScript

```
db.firstCollection.deleteMany({
  $and: [
    { age: { $gte: 23 } },
    { age: { $lte: 55 } },
    { city: "nalkheda" }
  ]
})
```

Lekin `$and` ki zarurat nahi bhi hai

MongoDB me ye same hai:

</> JavaScript

```
db.firstCollection.deleteMany({
  age: {
    $gte: 23,
    $lte: 55
  },
  city: "nalkheda"
})
```

Top-level fields automatically **AND** hote hain.

2. \$or ke saath deleteMany()

city Indore **OR** Bhopal wale documents delete karo.

</> JavaScript

```
db.firstCollection.deleteMany({
  $or: [
    { city: "Indore" },
    { city: "Bhopal" }
  ]
})
```

3. \$in ke saath deleteMany()

City Indore, Bhopal ya Ujjain me se koi bhi ho.

</> JavaScript

```
db.firstCollection.deleteMany({  
  city: {  
    $in: ["Indore", "Bhopal", "Ujjain"]  
  }  
})
```

Ye basically:

```
city = Indore  
OR  
city = Bhopal  
OR  
city = Ujjain
```



5. \$and + \$or

Suppose:

age 23+ **AND** city Indore **OR** Bhopal

</> JavaScript

```
db.firstCollection.deleteMany({
  $and: [
    { age: { $gte: 23 } },
    {
      $or: [
        { city: "Indore" },
        { city: "Bhopal" }
      ]
    }
  ]
})
```


`deleteOne()` vs `deleteMany()`

`deleteOne()`

- Sirf 1 document delete

`deleteMany()`

- Saare matched documents delete

2 important examples

1) One employee delete

 JavaScript

```
db.employees.deleteOne({ name: "Tarun Patidar" })
```



om/c/6a50b8ef-ec10-83ee-929e-0780ef2b4e4f

</> JavaScript

```
db.collectionName.updateOne(  
  { condition },  
  { $rename: { "oldFieldName": "newFieldName" } }  
)
```

Example 1: single document me field rename

mobile field ko **phone** banana:

</> JavaScript

```
db.students.updateOne(  
  { name: "Rahul Sharma" },  
  { $rename: { "mobile": "phone" } }  
)
```

Meaning: Rahul Sharma wale document me **mobile** field ka naam **phone** ho jayega.

Example 2: sab documents me field rename

Agar poori collection me `mobile` ko `phone` banana ho:

JavaScript

```
db.students.updateMany(  
  {},  
  { $rename: { "mobile": "phone" } }  
)
```

Tumhare database ke useful examples

employees me `post` → `designation`

JavaScript

```
db.employees.updateMany(  
  {},  
  { $rename: { "post": "designation" } }  
)
```



'6a50b8ef-ec10-83ee-929e-0780ef2b4e4f'

1) Update Array

Array me new value add → `$push`

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $push: { tags: "backend" } }  
)
```

Meaning: `tags` array me `"backend"` add hoga.

Array me multiple values add → `$push + $each`

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $push: { tags: { $each: ["nodejs", "express"] } }  
)
```



670a30b0e1-ec10-03ee-929e-0700e12b4e41

Embedded document me naya object add

JavaScript

```
db.socialPosts.updateOne(
  { _id: 1 },
  {
    $push: {
      comments: { user: "Tarun", msg: "Excellent" }
    }
  }
)
```

Embedded document remove karna

JavaScript

```
db.socialPosts.updateOne(
  { _id: 1 },
  { $pull: { comments: { user: "Rahul" } } }
)
```



2) Embedded Document Update

Tumhare `socialPosts` me `comments` embedded documents ka array hai:

JavaScript

```
comments: [  
  { user: "Rahul", msg: "Nice" },  
  { user: "Priya", msg: "Good" }  
]
```

Embedded document ka field update

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $set: { "comments.0.msg": "Very Nice" } }  
)
```



:/6a50b8ef-ec10-83ee-929e-0780ef2b4e4f

Array se value remove → \$pull

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $pull: { tags: "mongodb" } }  
)
```

Meaning: tags array se "mongodb" remove ho jayega.

Array me ek specific index update

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $set: { "tags.0": "database" } }  
)
```



Meaning: tags array ka first element update hoga.

3 Most Important Examples

1) Tag add

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $push: { tags: "interview" } }  
)
```

2) First comment message update

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $set: { "comments.0.msg": "Awesome" } }  
)
```



711/0/04300001-0010-0000-0000-070001207071

Example: first tag remove

</> JavaScript

```
db.socialPosts.updateOne(
  { _id: 1 },
  { $pop: { tags: -1 } }
)
```

2) Embedded document array se remove → \$pull

Agar embedded documents ka array ho, jaise:

</> JavaScript

```
comments: [
  { user: "Rahul", msg: "Nice" },
  { user: "Priya", msg: "Good" }
]
```



`$pop` array ke start ya end se element remove karta hai.

- `1` → last element remove
- `-1` → first element remove

Syntax

JavaScript

```
db.collectionName.updateOne(  
  { condition },  
  { $pop: { arrayField: 1 } } // last remove  
)
```

Example: last tag remove

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $pop: { tags: 1 } }  
)
```



Important difference

`$pop`

- Sirf **first/last element** remove karta hai
- Specific value/object target nahi karta

`$pull`

- **Matching value ya object** remove karta hai
- Arrays aur embedded document arrays dono me useful

Important difference

`findOne()`

- single document return karta hai

`find()`

- cursor return karta hai / multiple documents de sakta hai
-

Most important examples

1) Ek user find karna

 JavaScript

```
db.users.findOne({ phone: "9876543210" })
```



2) Indore ke saare users

</> JavaScript

```
db.users.find({ "address.city": "Indore" })
```

3) Age 22 wale users

</> JavaScript

```
db.users.find({ age: 22 })
```

4) Active users

</> JavaScript

```
db.users.find({ isActive: true })
```

Nested / Embedded field read

Embedded document ke field ko **dot notation** se access karte hain:

⌘ JavaScript

```
db.users.find({ "address.city": "Bhopal" })
```

Array field read

Agar array me "MongoDB" skill ho:

⌘ JavaScript

```
db.users.find({ skills: "MongoDB" })
```

Projection (sirf kuch fields dikhani ho)

Name aur phone hi chahiye

JavaScript

```
db.users.find(  
  { isActive: true },  
  { name: 1, phone: 1, _id: 0 }  
)
```

JavaScript

```
db.users.findOne({ name: "Tarun Patidar" })  
  
db.users.find({ age: 22 })  
  
db.users.find({ "address.city": "Indore" })  
  
db.users.find({ skills: "MongoDB" })  
  
db.users.find({}, { name: 1, phone: 1, _id: 0 })
```

2) `countDocuments()`

Total documents count

JavaScript

```
db.students.countDocuments()
```

Condition ke saath count

JavaScript

```
db.students.countDocuments({ city: "Indore" })
```

Example

JavaScript

```
db.employees.countDocuments({ department: "IT" })
```



JavaScript

```
db.students.find().limit(5)
```

Salary वाले first 3 employees

JavaScript

```
db.employees.find({}, { name: 1, salary: 1, _id: 0 }).limit(3)
```

4) skip()

First 5 skip karke next documents

JavaScript

```
db.students.find().skip(5)
```



Pagination style: first 10 skip, next 5

</> JavaScript

```
db.students.find().skip(10).limit(5)
```

5) `sort()`

Ascending sort by fees

</> JavaScript

```
db.students.find().sort({ fees: 1 })
```

Descending sort by salary

</> JavaScript

```
db.employees.find().sort({ salary: -1 })
```

Multiple field sort

⌘ JavaScript

```
db.employees.find().sort({ city: 1, salary: -1 })
```

6) sort + limit

Top 5 highest salary employees

⌘ JavaScript

```
db.employees.find({}, { name: 1, salary: 1, _id: 0 })  
.sort({ salary: -1 })  
.limit(5)
```

Lowest 3 fee students

JavaScript

```
db.students.find({}, { name: 1, fees: 1, _id: 0 })  
.sort({ fees: 1 })  
.limit(3)
```

7) Single document field read

Example: `students`, `employees`, `products` me simple fields

Students from Indore

JavaScript

```
db.students.find({ city: "Indore" })
```

11) Projection on embedded fields

Sirf title aur comments user dekhna

JavaScript

```
db.posts.find(  
  {},  
  { title: 1, "comments.user": 1, _id: 0 }  
)
```

12) Array size check

Jis post me exactly 3 tags ho

JavaScript

```
db.posts.find({ tags: { $size: 3 } })
```

:/ba50b8e7-ec10-83ee-929e-0780e72b4e4f

Delivered orders ka count

</> JavaScript

```
db.orders.countDocuments({ status: "Delivered" })
```

Latest-like top 5 posts by likes

</> JavaScript

```
db.posts.find({}, { title: 1, likes: 1, _id: 0 })  
.sort({ likes: -1 })  
.limit(5)
```

Highest fee courses

</> JavaScript

```
db.courses.find({}, { courseName: 1, fees: 1, _id: 0 })  
.sort({ fees: -1 })  
.limit(5)
```



```
JavaScript
```

```
use companyDB
```

```
db.students.countDocuments()
```

```
db.students.countDocuments({ city: "Indore" })
```

```
db.students.find().limit(5)
```

```
db.students.find().skip(5)
```

```
db.students.find().skip(10).limit(5)
```

```
db.students.find().sort({ fees: 1 })
```

```
db.employees.find().sort({ salary: -1 })
```

```
db.posts.find({ tags: "mongodb" })
```

```
db.posts.find({ "comments.user": "Rahul" })
```

```
db.posts.find({ "comments.msg": "Good" })
```

```
db.posts.find({}, { title: 1, "comments.user": 1, _id: 0 })
```

```
db.posts.find({ tags: { $size: 3 } })
```



A) Single / normal field

city = Indore

JavaScript

```
db.students.find({ city: { $eq: "Indore" } })
```

status = Delivered

JavaScript

```
db.orders.find({ status: { $eq: "Delivered" } })
```


B) Array field

`posts.tags` array hai:

`</>` JavaScript

```
tags: ["mongodb","nodejs","database"]
```

Jis post ke tags me `"mongodb"` ho

`</>` JavaScript

```
db.posts.find({ tags: { $eq: "mongodb" } })
```

C) Embedded document / nested field

Agar nested field access karna ho to **dot notation** use hota hai.

Jis comment ka msg = "Nice"

</> JavaScript

```
db.posts.find({ "comments.msg": { $eq: "Nice" } })
```

D) Embedded array (array of objects)

Jis post me user = Rahul comment ho

</> JavaScript

```
db.posts.find({ "comments.user": { $eq: "Rahul" } })
```

A) Single field

⟨/⟩ JavaScript

```
db.students.find({ city: { $ne: "Indore" } })
```

⟨/⟩ JavaScript

```
db.orders.find({ status: { $ne: "Delivered" } })
```

B) Array field

Jis post me "mongodb" tag na ho

⟨/⟩ JavaScript

```
db.posts.find({ tags: { $ne: "mongodb" } })
```

C) Embedded document field

Jis post me comment msg "Good" na ho

JavaScript

```
db.posts.find({ "comments.msg": { $ne: "Good" } })
```

D) Embedded array

Jis post me Rahul user wala comment na ho

JavaScript

```
db.posts.find({ "comments.user": { $ne: "Rahul" } })
```

A) Single field

fees > 70000

❏ JavaScript

```
db.students.find({ fees: { $gt: 70000 } })
```

salary > 100000

❏ JavaScript

```
db.employees.find({ salary: { $gt: 100000 } })
```

B) Array field

Jis post me `tags` array ka size compare directly `$gt` se nahi hota

Array ke size ke liye `$size` use hota hai, comparison operator नहीं.

✗ Direct नहीं:

⌞ JavaScript

```
db.posts.find({ tags: { $gt: 2 } }) // meaningful array size query nahi
```

Correct idea:

⌞ JavaScript

```
db.posts.find({ tags: { $size: 3 } })
```

Important: normal array values par `$eq/$ne` common hai.

Numeric array ho to `$gt/$lt` meaningful hota hai but tumhare `posts.tags` string array hai.



A) Single field

⌘ JavaScript

```
db.students.find({ age: { $gte: 24 } })
```

⌘ JavaScript

```
db.products.find({ rating: { $gte: 4 } })
```

B) Array field

String array me practical use कम है, but equality-style matching hota hai.

C) Embedded document field

likes $\geq$ 500 वाले posts

</> JavaScript

```
db.posts.find({ likes: { $gte: 500 } })
```

D) Embedded array

Agar comments me numeric field hota to:

</> JavaScript

```
// concept  
// db.posts.find({ "comments.score": { $gte: 5 } })
```

A) Single field

</> JavaScript

```
db.products.find({ price: { $lt: 30000 } })
```

</> JavaScript

```
db.courses.find({ fees: { $lt: 25000 } })
```

B) Array field

`tags` string array hai, so `$lt` practically useful nahi.

C) Embedded document field

likes < 200 वाले posts

❏ JavaScript

```
db.posts.find({ likes: { $lt: 200 } })
```

D) Embedded array

Conceptually:

❏ JavaScript

```
// db.posts.find({ "comments.score": { $lt: 3 } })
```

A) Single field

⌘ JavaScript

```
db.students.find({ age: { $lte: 20 } })
```

⌘ JavaScript

```
db.products.find({ rating: { $lte: 2 } })
```

B) Array field

Again, string arrays me `$lte` practical nahi.

C) Embedded document field

likes <= 100

JavaScript

```
db.posts.find({ likes: { $lte: 100 } })
```

D) Embedded array

Concept same:

JavaScript

```
// db.posts.find({ "comments.score": { $lte: 2 } })
```

7) Range query (single field)

Students fees between 50000 and 70000

JavaScript

```
db.students.find({  
  fees: { $gte: 50000, $lte: 70000 }  
})
```

Employees salary between 60000 and 120000

JavaScript

```
db.employees.find({  
  salary: { $gt: 60000, $lt: 120000 }  
})
```

8) Range on post likes

likes between 200 and 500

JavaScript

```
db.posts.find({  
  likes: { $gte: 200, $lte: 500 }  
})
```

9) Embedded array real examples from your posts

Rahul comment वाले posts

JavaScript

```
db.posts.find({ "comments.user": "Rahul" })
```



A) Students from Indore, Bhopal, Delhi

❏ JavaScript

```
db.students.find({  
  city: { $in: ["Indore", "Bhopal", "Delhi"] }  
})
```

B) Students not from Indore, Bhopal

❏ JavaScript

```
db.students.find({  
  city: { $nin: ["Indore", "Bhopal"] }  
})
```

C) Employees from IT or HR department

⌘ JavaScript

```
db.employees.find({  
  department: { $in: ["IT", "HR"] }  
})
```

D) Employees not in Sales/Admin

⌘ JavaScript

```
db.employees.find({  
  department: { $nin: ["Sales", "Admin"] }  
})
```


2) Array Field

Tumhare `posts` collection me:

</> JavaScript

```
tags: ["mongodb", "nodejs", "database"]
```

A) Posts jinke tags me mongodb ya nodejs ho

</> JavaScript

```
db.posts.find({  
  tags: { $in: ["mongodb", "nodejs"] }  
})
```

Meaning: agar `tags` array me inme se **koi bhi ek value** present hai, document match hoga.

B) Posts jinke tags me `java` ya `python` na ho

JavaScript

```
db.posts.find({
  tags: { $nin: ["java", "python"] }
})
```

3) Embedded / Nested Field

Tumhare `posts.comments` me:

JavaScript

```
comments: [
  { user: "Rahul", msg: "Nice" },
  { user: "Priya", msg: "Good" }
]
```



A) Jis post me comment user Rahul ya Priya ho

⟨/⟩ JavaScript

```
db.posts.find({  
  "comments.user": { $in: ["Rahul", "Priya"] }  
})
```

B) Jis post me comment msg Nice ya Good ho

⟨/⟩ JavaScript

```
db.posts.find({  
  "comments.msg": { $in: ["Nice", "Good"] }  
})
```

0780ef2b4e4f

C) Jis post me comment user Amit ya Tarun na ho

</> JavaScript

```
db.posts.find({  
  "comments.user": { $nin: ["Amit", "Tarun"] }  
})
```

4) Embedded Array / Array of Embedded Documents

`comments` embedded array hai, aur uske andar `user`, `msg` fields hain.

A) Rahul / Priya comment वाले posts

</> JavaScript

```
db.posts.find({  
  "comments.user": { $in: ["Rahul", "Priya"] }  
})
```



5) Date + \$in

Agar exact multiple dates match karni ho:

JavaScript

```
db.orders.find({
  orderDate: {
    $in: [
      new Date("2024-01-02"),
      new Date("2024-02-03")
    ]
  }
})
```

7) Combined with other conditions

IT/HR employees from Indore

JavaScript

```
db.employees.find({
  department: { $in: ["IT", "HR"] },
  city: "Indore"
})
```

Rating 4/5 products from Delhi or Mumbai

JavaScript

```
db.products.find({
  rating: { $in: [4, 5] },
  city: { $in: ["Delhi", "Mumbai"] }
})
```

</> JavaScript

```
db.collectionName.find({
  $and: [
    { condition1 },
    { condition2 }
  ]
})
```

Single field example

Indore ke students jinki age 22 ho

</> JavaScript

```
db.students.find({
  $and: [
    { city: "Indore" },
    { age: 22 }
  ]
})
```

Employees jinka department IT ho aur salary > 100000

</> JavaScript

```
db.employees.find({
  $and: [
    { department: "IT" },
    { salary: { $gt: 100000 } }
  ]
})
```

Array field example

`posts.tags` array hai.

Jis post me mongodb tag ho aur likes > 300 ho

</> JavaScript

```
db.posts.find({
  $and: [
    { tags: "mongodb" },
    { likes: { $gt: 300 } }
  ]
})
```


Embedded array example

Rahul comment aur likes > 200 वाले posts

`</>` JavaScript

```
db.posts.find({
  $and: [
    { "comments.user": "Rahul" },
    { likes: { $gt: 200 } }
  ]
})
```

Students jo Indore ya Bhopal se ho

</> JavaScript

```
db.students.find({
  $or: [
    { city: "Indore" },
    { city: "Bhopal" }
  ]
})
```

Products jinki rating 5 ho ya price < 20000 ho

</> JavaScript

```
db.products.find({
  $or: [
    { rating: 5 },
    { price: { $lt: 20000 } }
  ]
})
```

Embedded array example

Rahul ya Priya comment वाले posts

JavaScript

```
db.posts.find({
  $or: [
    { "comments.user": "Rahul" },
    { "comments.user": "Priya" }
  ]
})
```

⌞ JavaScript

```
db.collectionName.find({  
  fieldName: { $not: { operator } }  
})
```

Single field example

Students jinki age 22 nahi ho

⌞ JavaScript

```
db.students.find({  
  age: { $not: { $eq: 22 } }  
})
```

Products jinka rating 5 nahi ho

JavaScript

```
db.products.find({  
  rating: { $not: { $eq: 5 } }  
})
```

Employees jinki salary 100000 se zyada nahi ho

JavaScript

```
db.employees.find({  
  salary: { $not: { $gt: 100000 } }  
})
```

Jis post me mongodb tag na ho

</> JavaScript

```
db.posts.find({  
  tags: { $not: { $eq: "mongodb" } }  
})
```

Embedded array example

Jis post me Rahul comment na ho

</> JavaScript

```
db.posts.find({  
  "comments.user": { $not: { $eq: "Rahul" } }  
})
```

4) \$nor

Meaning

Di hui **saari conditions false** honi chahiye.

Matlab **none of the conditions should be true.**

Syntax

⌞ JavaScript

```
db.collectionName.find({  
  $nor: [  
    { condition1 },  
    { condition2 }  
  ]  
})
```

Students jo na Indore ke ho na Bhopal ke

</> JavaScript

```
db.students.find({
  $nor: [
    { city: "Indore" },
    { city: "Bhopal" }
  ]
})
```

Products jinki rating 5 bhi na ho aur price < 20000 bhi na ho

</> JavaScript

```
db.products.find({
  $nor: [
    { rating: 5 },
    { price: { $lt: 20000 } }
  ]
})
```



Embedded array example

Jis post me na Rahul comment ho na Priya comment ho

`</>` JavaScript

```
db.posts.find({
  $nor: [
    { "comments.user": "Rahul" },
    { "comments.user": "Priya" }
  ]
})
```

```
use companyDB

// AND
db.students.find({ $and: [ { city: "Indore" }, { age: 22 } ] })
db.posts.find({ $and: [ { tags: "mongodb" }, { likes: { $gt: 300 } } ] })

// OR
db.students.find({ $or: [ { city: "Indore" }, { city: "Bhopal" } ] })
db.posts.find({ $or: [ { "comments.user": "Rahul" }, { "comments.user": "Priya" } ] })

// NOT
db.students.find({ age: { $not: { $eq: 22 } } })
db.posts.find({ tags: { $not: { $eq: "mongodb" } } })

// NOR
db.students.find({ $nor: [ { city: "Indore" }, { city: "Bhopal" } ] })
db.posts.find({ $nor: [ { tags: "mongodb" }, { tags: "nodejs" } ] })
```

XOR Example 1

Students: city = Indore XOR age = 22

Matlab:

- ya to city Indore ho
- ya age 22 ho
- **but dono saath me na ho**

JavaScript

```
db.students.find({
  $or: [
    { city: "Indore", age: { $ne: 22 } },
    { city: { $ne: "Indore" }, age: 22 }
  ]
})
```

XOR Example 2

Products: rating = 5 XOR city = Delhi

⟨/⟩ JavaScript

```
db.products.find({
  $or: [
    { rating: 5, city: { $ne: "Delhi" } },
    { rating: { $ne: 5 }, city: "Delhi" }
  ]
})
```

XOR Example 3

Posts: tag mongodb XOR likes > 300

⟨/⟩ JavaScript

```
db.posts.find({
  $or: [
    { tags: "mongodb", likes: { $lte: 300 } },
    { tags: { $ne: "mongodb" }, likes: { $gt: 300 } }
  ]
})
```

XOR Example 4

Posts: Rahul comment XOR likes > 400

JavaScript

```
db.posts.find({
  $or: [
    { "comments.user": "Rahul", likes: { $lte: 400 } },
    { "comments.user": { $ne: "Rahul" }, likes: { $gt: 400 } }
  ]
})
```

MongoDB XNOR Examples

XNOR Example 1

Students: city = Indore XNOR age = 22

Matlab:

- dono true
ya
- dono false

JavaScript

```
db.students.find({
  $or: [
    { city: "Indore", age: 22 },
    { city: { $ne: "Indore" }, age: { $ne: 22 } }
  ]
})
```

XNOR Example 2

Products: rating = 5 XNOR city = Delhi

JavaScript

```
db.products.find({
  $or: [
    { rating: 5, city: "Delhi" },
    { rating: { $ne: 5 }, city: { $ne: "Delhi" } }
  ]
})
```

XNOR Example 3

Posts: Rahul comment XNOR likes > 400

JavaScript

```
db.posts.find({
  $or: [
    { "comments.user": "Rahul", likes: { $gt: 400 } },
    { "comments.user": { $ne: "Rahul" }, likes: { $lte: 400 } }
  ]
})
```

```
use companyDB
```

```
// XOR
```

```
db.students.find({  
  $or: [  
    { city: "Indore", age: { $ne: 22 } },  
    { city: { $ne: "Indore" }, age: 22 }  
  ]  
})
```

```
// XNOR
```

```
db.students.find({  
  $or: [  
    { city: "Indore", age: 22 },  
    { city: { $ne: "Indore" }, age: { $ne: 22 } }  
  ]  
})
```